Stack → Ex: Arranging the box with books
(LIFO) → Last In First Out
pop() → removes the top element
push() → adds element at top
peek() → returns top element
isEmpty() → return true/false



Concept:
"A stack is not just a container. It is useful when latest things handled first"

Monotonic stack

Task: arr = [2, 1, 5, 3, 4]
for every number, find next greater element
[5, 5, -1, 4, -1]

In brute force
→ look right, find first bigger element

Monotonic Stack
Is simply a stack where we maintain the elements in a particular order

Two forms
Monotonically increasing

1
2
4 ↑
7

Bottom → top is increasing

decreasing

9
6
4 ↑
2

decreasing

Next greater element 5 5 ↓ 4

[2, 1, 5, 3, 4]

Stack: [2], [1, 2], (5>1) pop(1)

check top element and pop

[5, 2], 5>2, pop(2)

[5]

[3, 5] → [(4>3) pop(3)

[4, 5]

-1 -1

(left to right) Traversal

```
for(int i=0; i<n; i++){
  while(!stack.isEmpty() &&
        arr[i] > stack.peek()){
    int element = stack.pop();
    map.add(element, arr[i];
  }
  stack.push(arr[i]);
}
while(stack.isEmpty()){
  int element = stack.pop();
  map.add(element, -1);
}
```

Four patterns →

Traversal Direction

Next greater
Next smaller } → ~~left to right~~ or right to left

↳ Increasing

Decreasing stack

Increasing Stack

Previous greater
Previous Smaller } → left to right

Date : _____

LC. 496

Next greater element (right to left) Traversal

[2, 1, 5, 3, 4]

map

Stack = 1. [4] (4, -1)

2. [3, 4] (3, 4)

3. 5 > 3 → [3, 4] → [4] 5 > 4 → [5] → [5]

Stack empty (5, -1)

4. 1 < 5 [1, 5] → (1, 5)

5. 2 2 > 1 [1, 5] → [5] → [2, 5] 2 < 5

(2, 5)

LC. 1475 (right to left)

Next Smaller element

[8, 4, 6, 2, 3]

result value is minus of next Smaller element

res = [4, 2, 4, 2, 3]

Stack

1. 3 [3]

map → direct array storing

{ (3, 0)

2. 2 ⇒ 2 < 3 so pop top element [2]

(2, 0)

3. 6 ⇒ 6 > 2 ⇒ add to stack [6, 2] → Top

(6, 2)

4. 4 ⇒ 4 < 6 ⇒ pop 6, 4 > 2, stop [4, 2] → Top

(4, 2)

5. 8 ⇒ 8 > 4 add to map with top element

(8, 4)

Date :

LC 901 → Left to right

Previous Smaller

class

Here in Stack we store array of 2 values

[price, span)

0 →

[100, 80, 60, 70, 60, 75, 85]

Stack

1. 100 => | 100,1 |

2. 80 < 100 => | 80,1 | 100,1 |

3. 60 < 80 => | 60,1 | 80,1 | 100,1 |

here

4. 70. 70 ≥ 60, top → add span

so pop 60, +1

70 < 80 so add to stack

70,2
80,1
100,1

5. 60, top 70 > 60 add to stack

60,1
70,2
80,1
100,1

6. 75, top 60 < 75

remove 60, add span of 60 to 75 => 2

70 < 75

pop 70, add span of 70 to 75 => 2+2 = 4

80 > 75 stop, add to stack

75,4
80,1
100,1

top 75

7. 85 => pop 75

1+4=5

top 80,

pop 80, add 1 => 5+1 = 6

Final stack:

85,6
100,1

Date :

```
Class StockSpanner {
    Stack <int[]> stack;
    public StockSpanner() {
        stack = new Stack<>();}

    public int next(int price) {
        int span = 1;
        while(!stack.isEmpty() &&
                stack.peek()[0] <= price){
            span += stack.pop()[1];
        }
        stack.push(new int[]{price, span});
        return span;
    }
}
```